

Modern C++ for Absolute Beginners: Course Syllabus

Comprehensive Curriculum Specification for AI Course Generation

1. Course Overview & Target Audience

This curriculum introduces foundational computing and modern C++ (C++17/20) to absolute beginners. It bridges theoretical hardware/memory concepts with practical software construction. Learners will gain fluency in compilation, strong typing, control flow, functions, memory references, modern STL containers (`std::vector`, `std::string`), and foundational object-oriented programming.

2. Learning Outcomes

Module Breakdown

Module 1: Getting Started & The C++ Compilation Model

Topics: What is C++? Interpreted vs Compiled languages. The preprocessor (`#include`), the `main()` function, namespaces (`std::`), and standard output with `std::cout`. Step-by-step walkthrough of writing, compiling, and running your first hello world program.

Module 2: Variables, Data Types & Basic Input

Topics: Fundamental primitive types (`int`, `float`, `double`, `char`, `bool`). Memory sizing with `sizeof()`. Variable initialization, `const` qualifiers, `auto` keyword, and receiving user input with `std::cin`. Common pitfall: uninitialized memory and buffer overflow.

Module 3: Control Flow & Decision Making

Topics: Boolean logic and relational operators. `if`, `else-if`, and `else` branching. Switch statements and fall-through behavior. Loops: `while`, `do-while`, and standard counting `for`-loops. Loop control with `break` and `continue`.

Module 4: Functions & Modular Programming

Topics: Function declaration vs definition. Parameters and return types. Function overloading. Scope, lifetime, and the call stack. Pass-by-value vs pass-by-reference (`&`). Using `const` references for efficiency.

Module 5: Strings and Dynamic Sequences (std::vector)

Topics: Modern text handling with std::string (concatenation, substr, size). Sequence containers with std::vector: push_back, size, indexing, and range-based for loops. Avoiding archaic raw C-style arrays in modern code.

Module 6: Pointers, References & The Memory Model

Topics: What is memory? Addresses and the address-of operator (&). Pointer declaration and dereferencing (*). nullptr vs garbage pointers. Stack memory allocation vs heap allocation using new and delete. Memory leak pitfalls and best practices.

Module 7: Introduction to Object-Oriented Programming

Topics: Custom data structures with struct vs class. Encapsulation: public vs private access specifiers. Member variables and member functions. Default, parameterized, and copy constructors. Basic resource management.

Module 8: Capstone Project & Final Assessment

Topics: Integration of functions, classes, vectors, and memory references. Hands-on coding lab: Building a text-based Student Record or Inventory Management Console Application. Comprehensive diagnostic assessment covering all course modules.

3. Pedagogical Methodology Recommendations

- Code Labs: Ensure coding exercises have starter boilerplate with main() and expected output comments.
- Stepper Lessons: Use step-by-step walkthroughs for compiler setup, the linking process, and memory tracing.
- Sequencer Challenges: Challenge students to order the execution sequence of nested loops, constructor calls, and the compilation pipeline.
- Scenario Lessons: Present debugging trade-offs (e.g. diagnosing a segmentation fault or memory leak).
- Cloze Fill-in-the-Blanks: Test core keywords (const, nullptr, static_cast, vector, template).